

GRAPHICS PROGRAMMING KNIGHTS



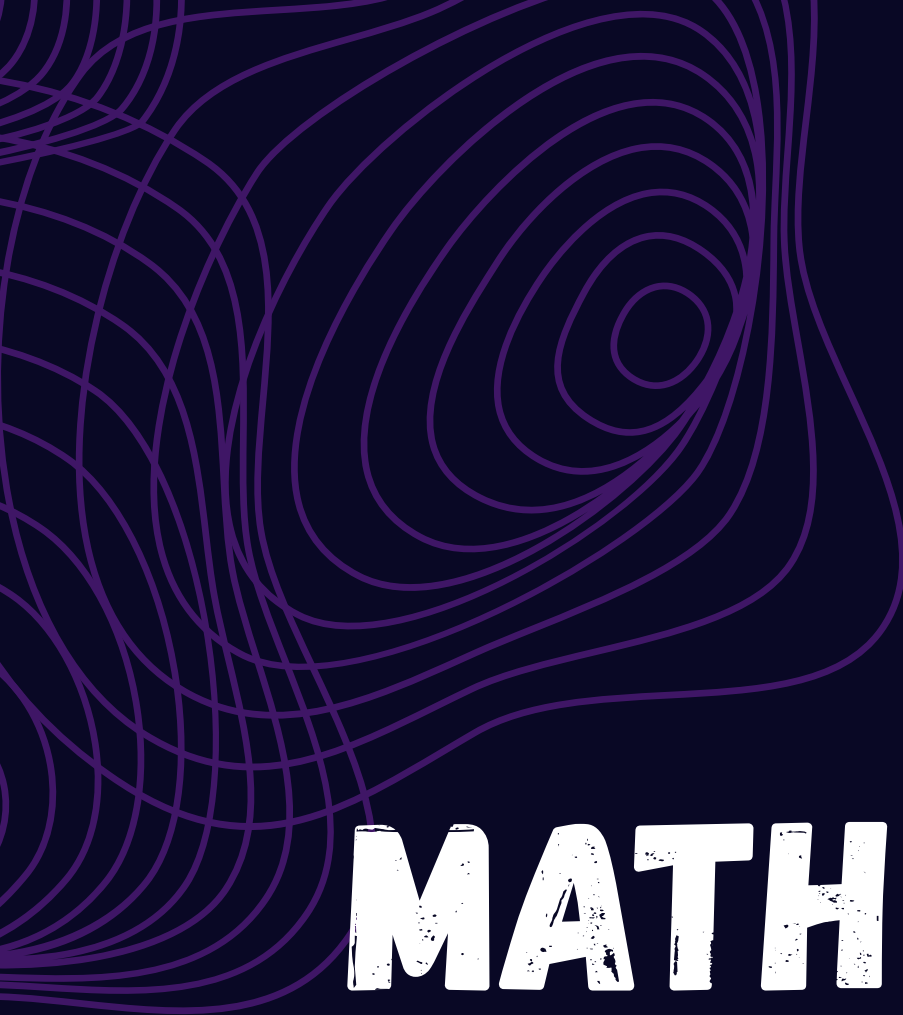
MATH FOR MAKING COOL THINGS APPEAR ON THE SCREEN WORKSHOP

An introductory way to immerse yourself into the
world of math in programming

Presented by: Alejandro Jaimes

WHY MATH?!
MATH IS DUMB!!!





MATH IN GRAPHICS, GAMES, AND MACHINE LEARNING

WHAT QUESTIONS DOES MATH ANSWER?

VIDEO GAMES

WHAT THE GAME NEEDS TO KNOW:

Did my character hit the opponent?

How does the enemy move to find you?

Who takes priority on a collision?

MATH BEHIND IT:

-Vectors → position and direction

-Dot product → distance

-Transformations → how does it move



WHAT QUESTIONS DOES MATH ANSWER?

MACHINE LEARNING

HOW DOES THE MACHINE LEARN?

How should I catalog this person?

Is this the right prediction?

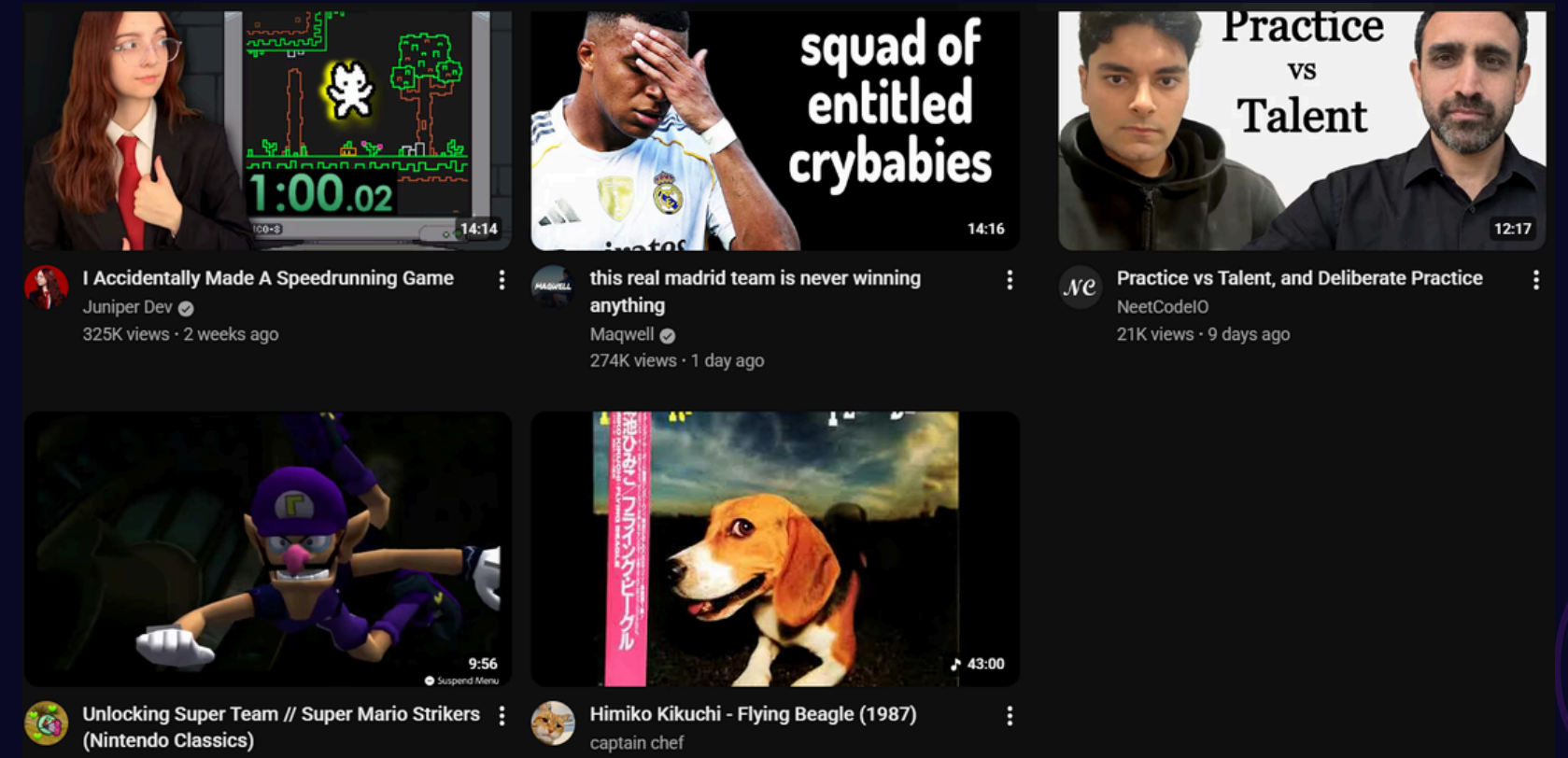
What is the pattern?

MATH BEHIND IT:

Vectors → Listings

Matrices → Store the data sets

Dot Product → How similar are these



WHAT QUESTIONS DOES MATH ANSWER

GRAPHICS

HOW DO YOU MAKE GRAPHICS?

How do I place objects?

How much light is needed?

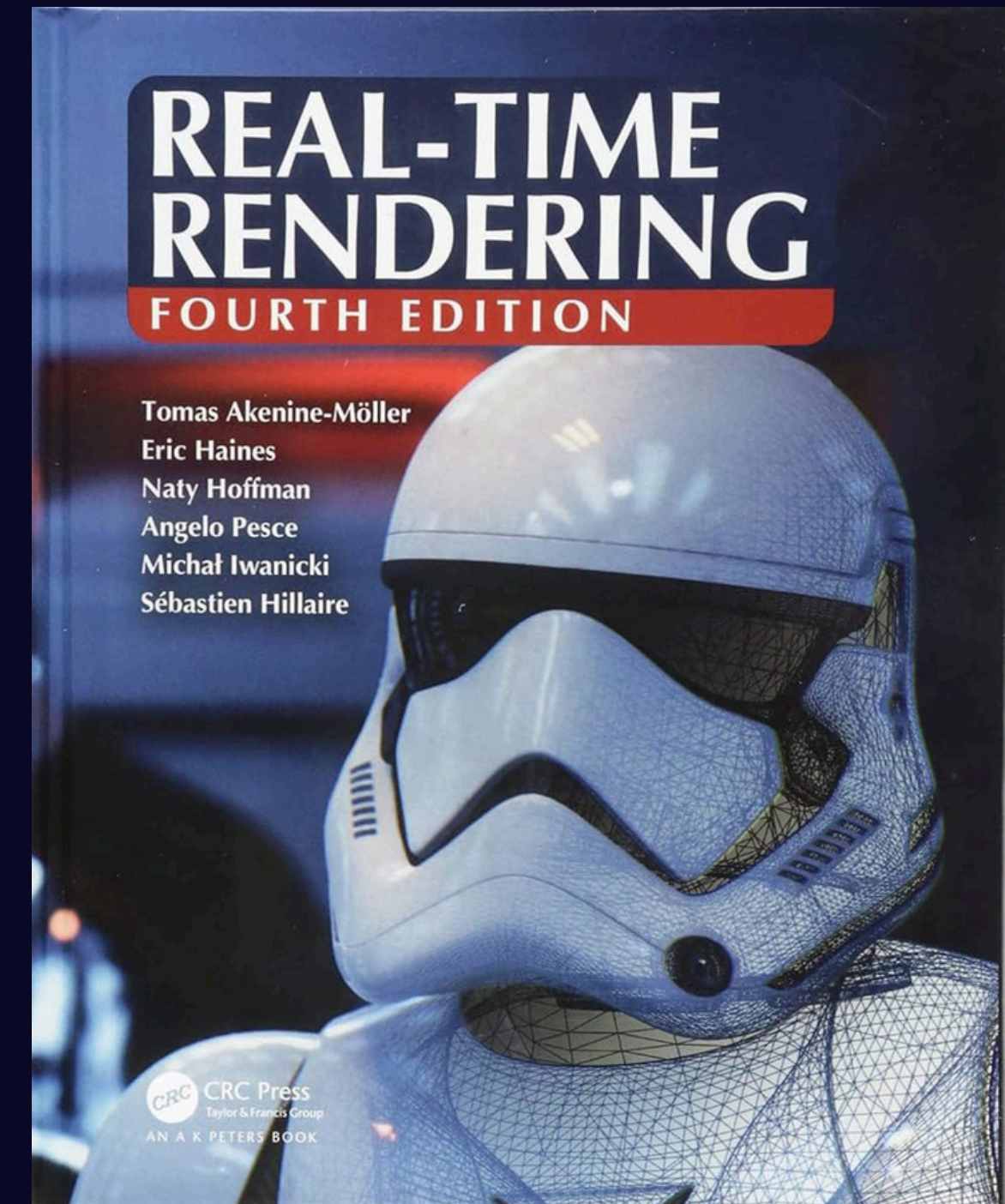
How should this move?

MATH BEHIND IT:

Vectors → Direction of light

Cross Product → Building the coordinate frames

Matrices → Transform 3D into a 2d screen



The background is a solid dark blue. In the top-left and bottom-right corners, there are decorative elements consisting of concentric, overlapping purple lines that form a swirling, organic pattern.

VECTORS AND PRODUCTS

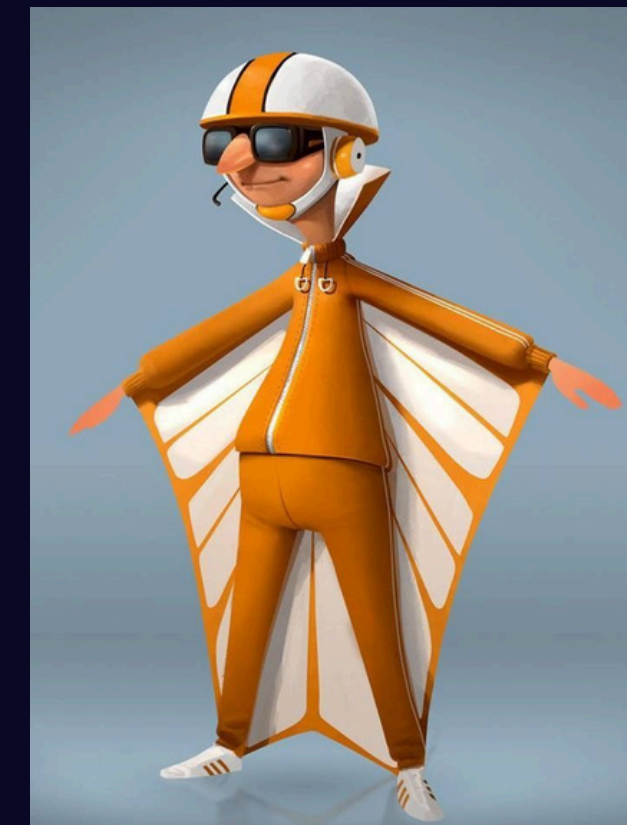
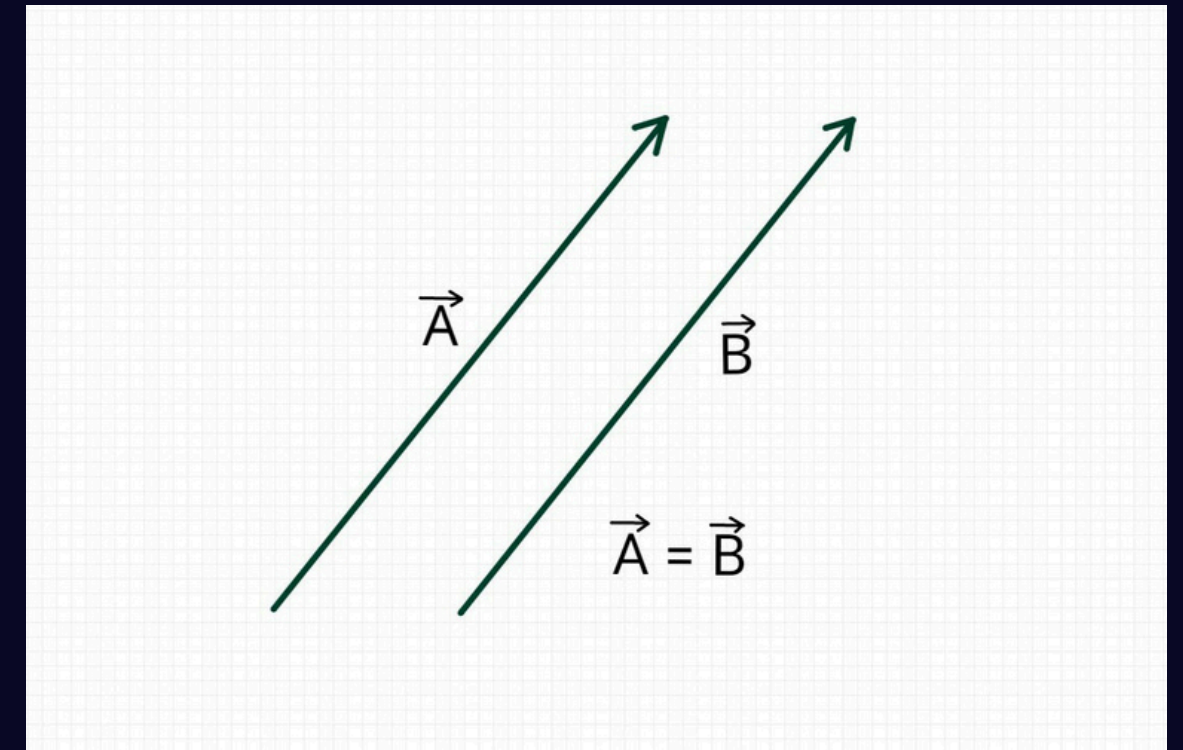
VECTORS

“Once you get vectors
everything makes sense”

In videogames, vectors are utilized to
correctly analyze movement that should
happen with magnitude and direction

In CG it helps with modeling and how the
light interacts with the environment

For ML, it is a bit different, they are
used to identify how the data is, what
it represents and the way something
relates to another thing learned



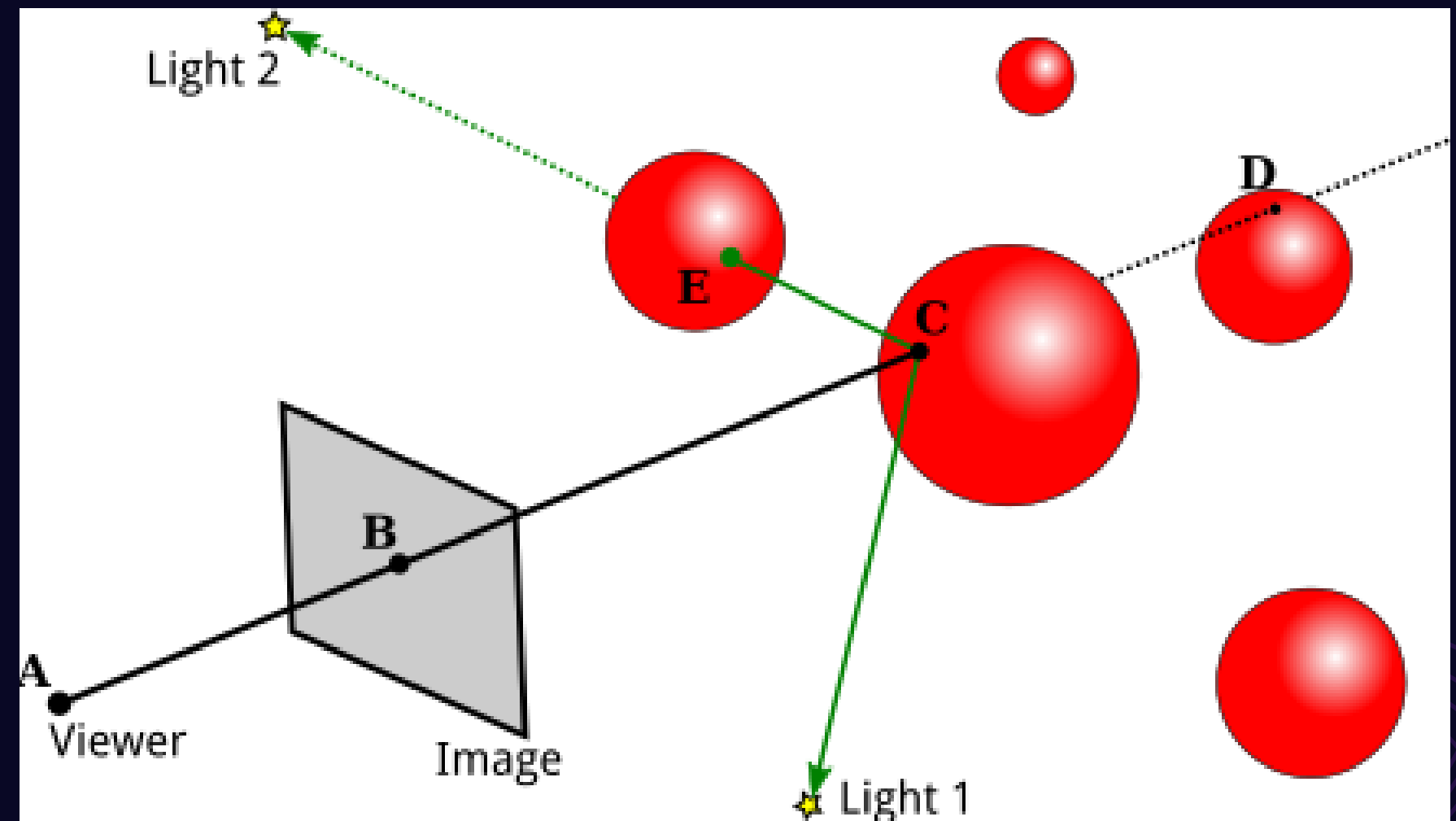
DOT PRODUCT AND CROSS PRODUCT

Dot and cross product are fundamental to the theory for CG and ML

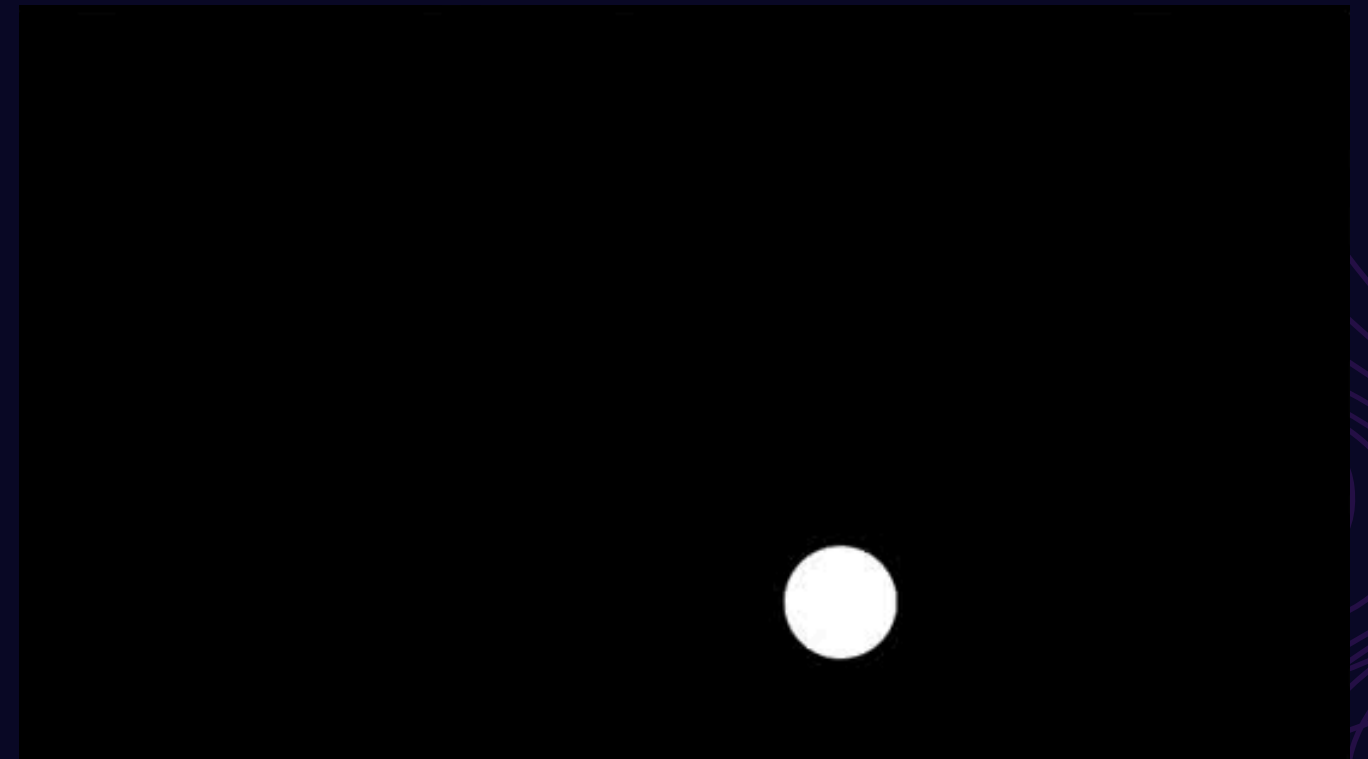
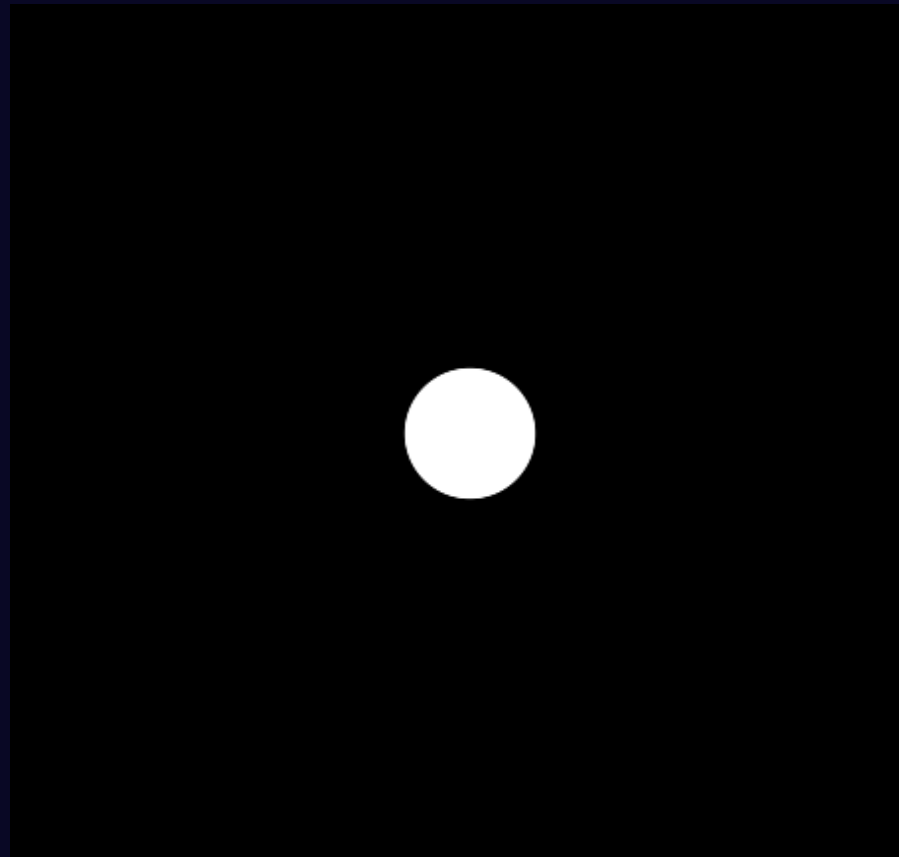
Both feature on a technique called Ray Tracing (pic on the right)

Dot product is more focused on ML by being a backbone of estimating total similarity

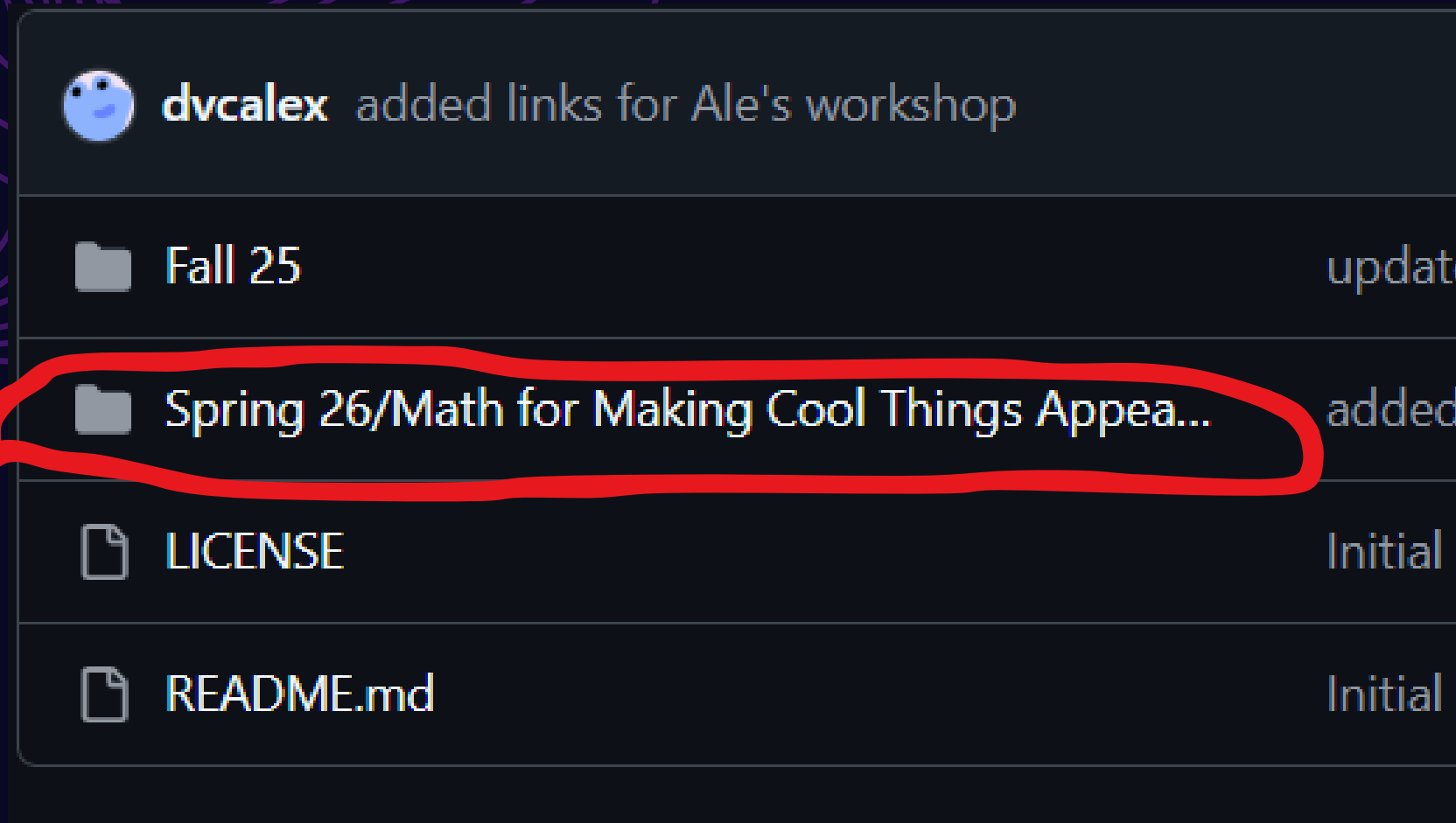
Meanwhile, cross product is extremely helpful for rendering and see good the object looks



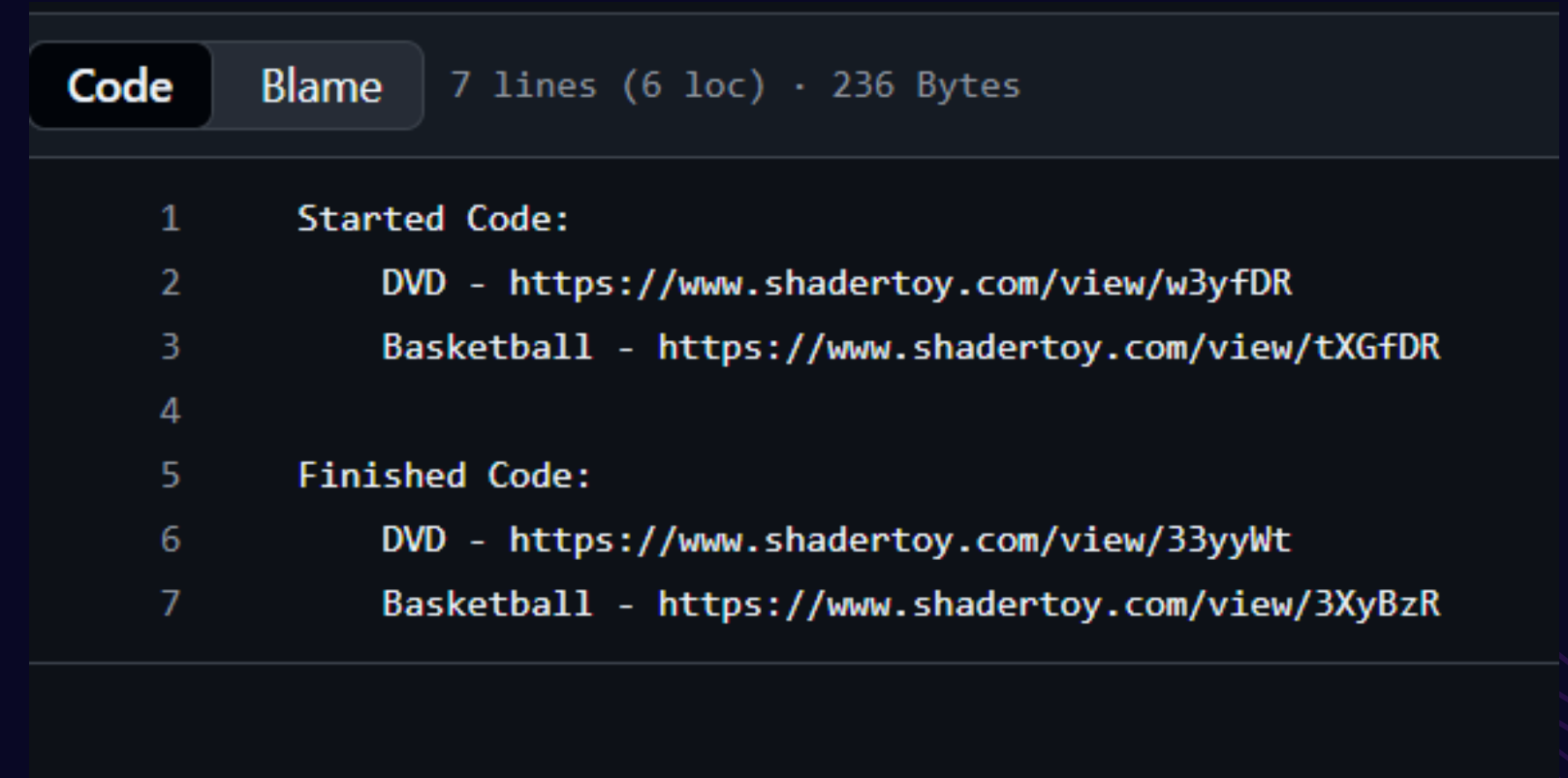
**LET'S CODE
SOMETHING COOL**



<https://github.com/GraphicsProgrammingKnights/Workshops/tree/main>
also putting this on math channel



select this one



copy the link for the one we
are doing

shadertoy.com

HERE IS THE FINAL CODE

```
Shader Inputs
1 float circle(vec2 uv, float r, float blur) {
2     float diameter = length(uv);
3     return smoothstep(r, r-blur, diameter);
4 }
5 vec2 velocity = vec2(1.0, 0.65);
6 void mainImage(out vec4 fragColor, in vec2 fragCoord) {
7
8     vec2 screenUV = fragCoord / iResolution.xy;
9     screenUV = screenUV * 2.0 - 1.0;
10
11     // drawing
12     vec2 drawing = screenUV;
13     drawing.x *= iResolution.x / iResolution.y;
14
15     float radius = 0.15;
16     float pixelWidth = 2.0 / iResolution.y;
17     float aspect = iResolution.x / iResolution.y;
```

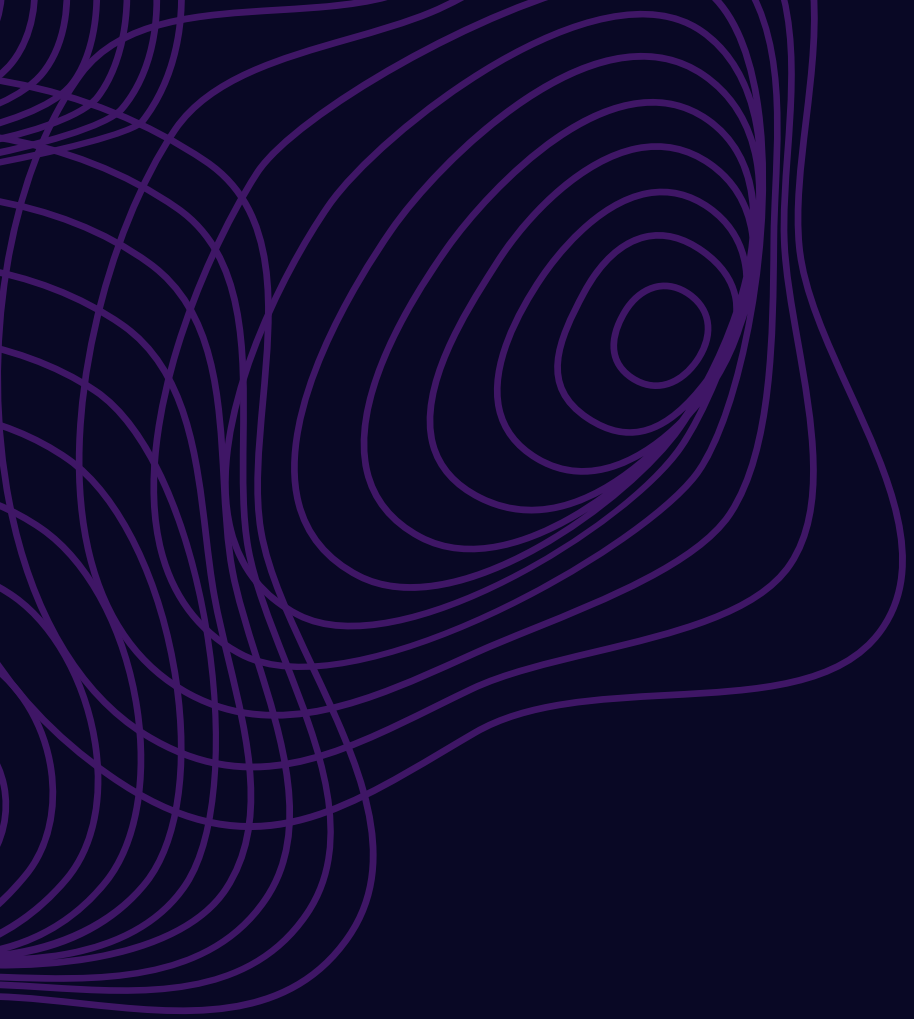
s://www.shadertoy.com/view/33yyWt

1/

1/26, 9:52 PM

aweosme

```
18     // bounds for bouncing region around origin
19     vec2 minBounds = vec2(-1.0 + radius, -1.0 + radius);
20     vec2 maxBounds = vec2( 1.0 - radius,  1.0 - radius);
21     vec2 range = (maxBounds - minBounds);
22     // shit actually moving
23     vec2 rawPos = velocity * iTime;
24
25     // moving but taking in consideration bounds
26     vec2 position = minBounds + abs(mod(rawPos - minBounds, range * 2.0)
27     - range);
28
29     // move circle
30     vec2 movingUV = drawing - vec2(position.x*aspect, position.y);
31
32     float bw = circle(movingUV, radius, pixelWidth);
33     fragColor = vec4(vec3(bw), 1.0);
34 }
```



MATRICES AND TRANSFORMATIONS

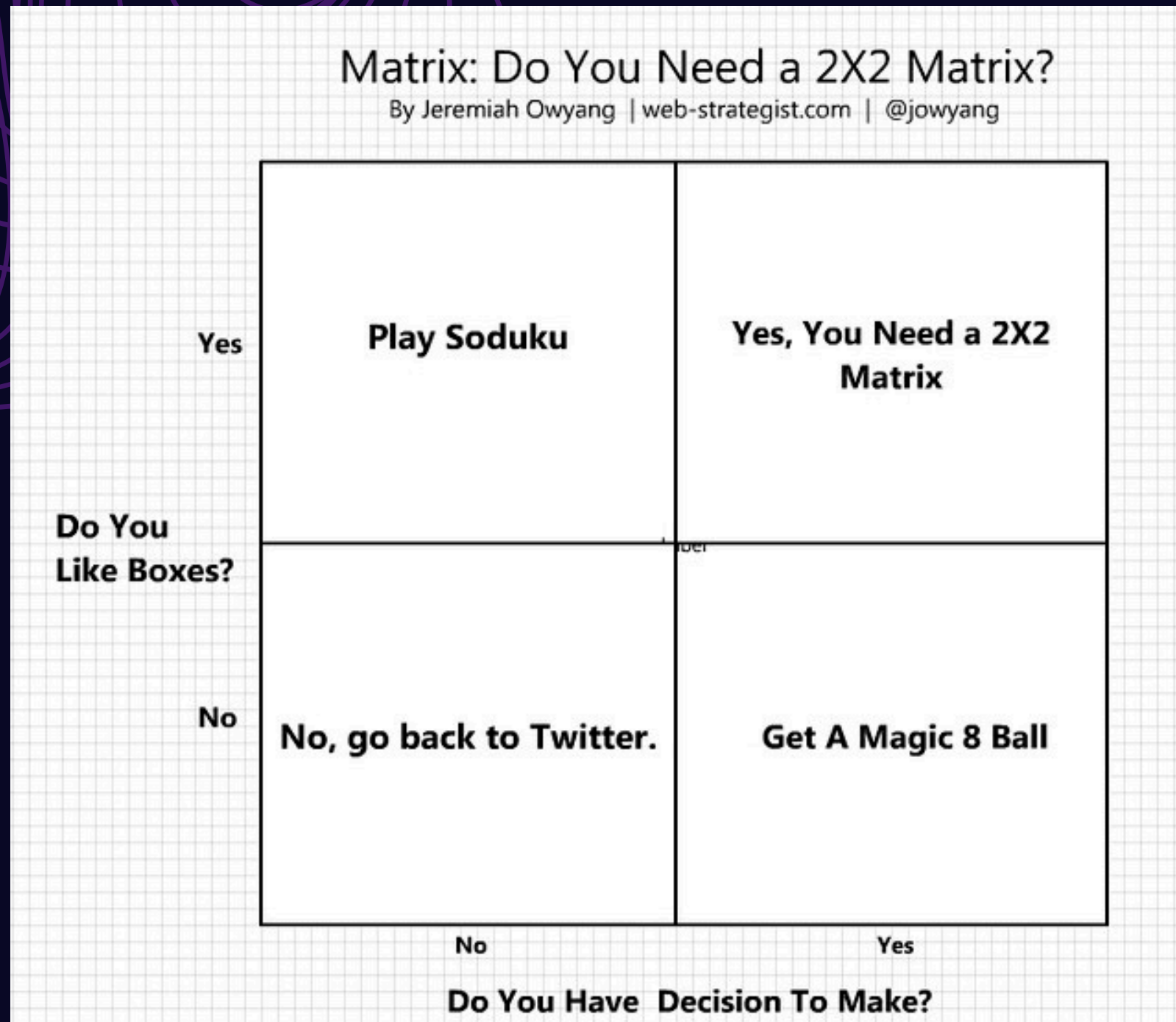
MATRICES

Matrices is alongside vectors the backbone for a lot of things we see today

They represent data very accurately, and in an organized matter

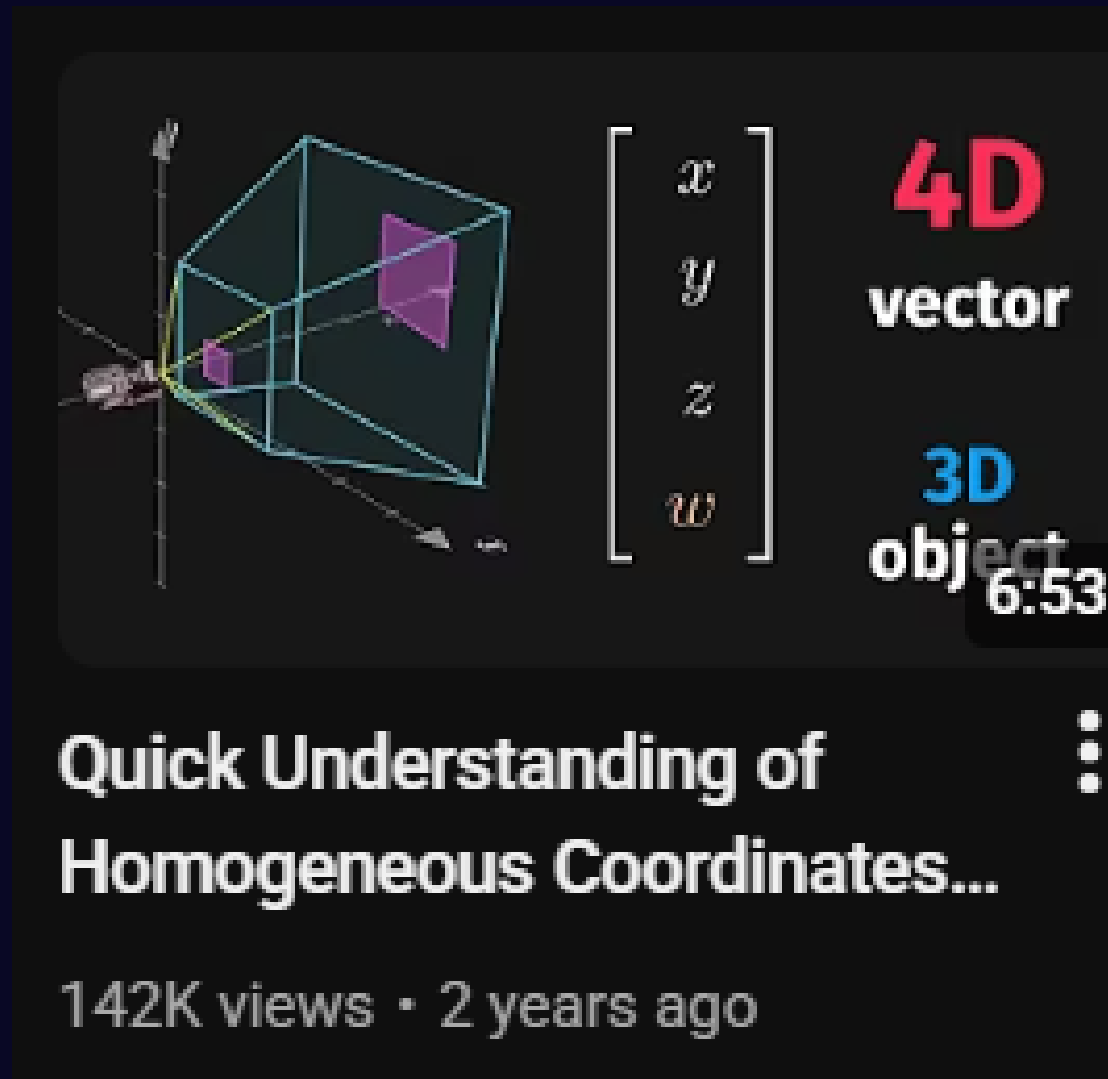
These are big for Computer Graphics by allowing a better way to visualize an object

For ML, it can serve as way a to weigh the biases the machine may have



HOMOGENOUS COORDINATES

LINK VIDEO LATER, WONT GO INTO MUCH DETAIL BECAUSE IT
IS AN EXTRA THING, ONLY IF TIME ALLOWS



By: Miolith
[https://youtu.be/o-xwmTODTUI?
si=rrUPCFGKqpW0krN8](https://youtu.be/o-xwmTODTUI?si=rrUPCFGKqpW0krN8)

TRANSFORMATIONS

THE INTENTIONALITY BEHIND

Transformations are where bugs become visible

Transformations are essential for controlling the exact behavior of the system

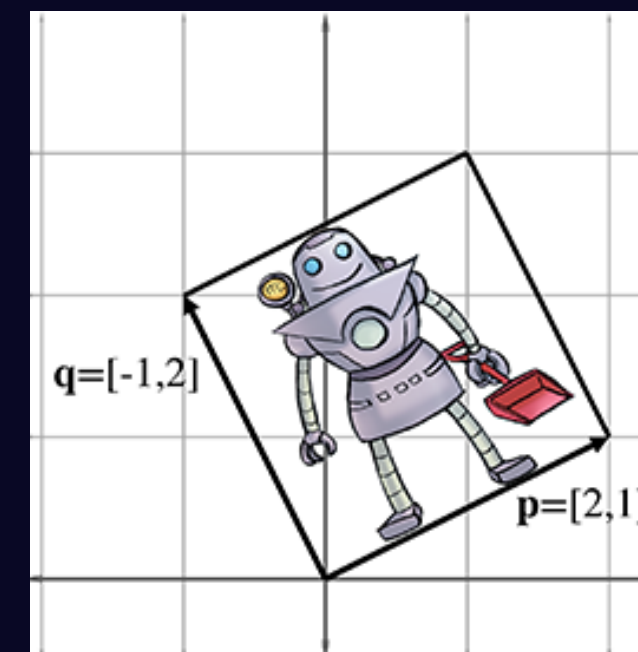
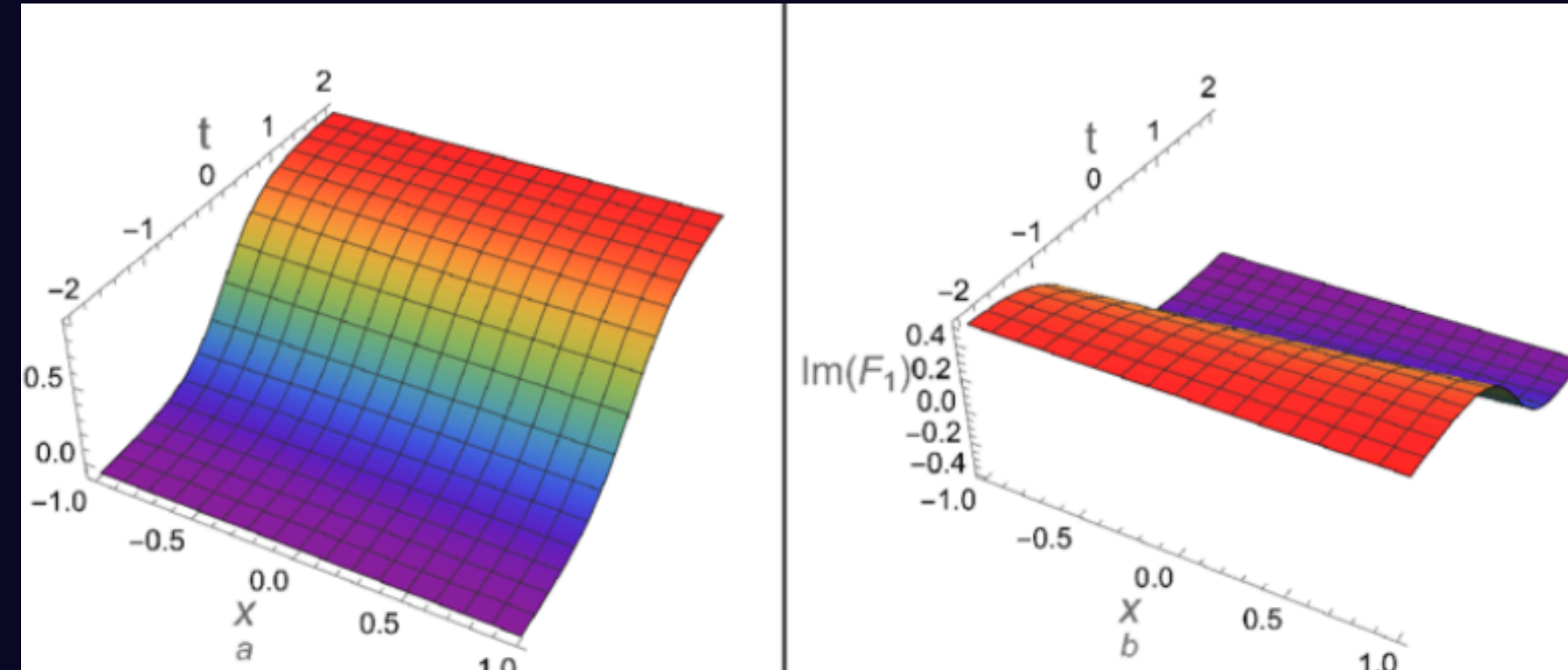
Common transformations:

Translations → where is it located

Rotation → where is it looking at

Scale → how big or small is this?

Reflection → can you mirror it?



REFLECTION

TRANSLATION

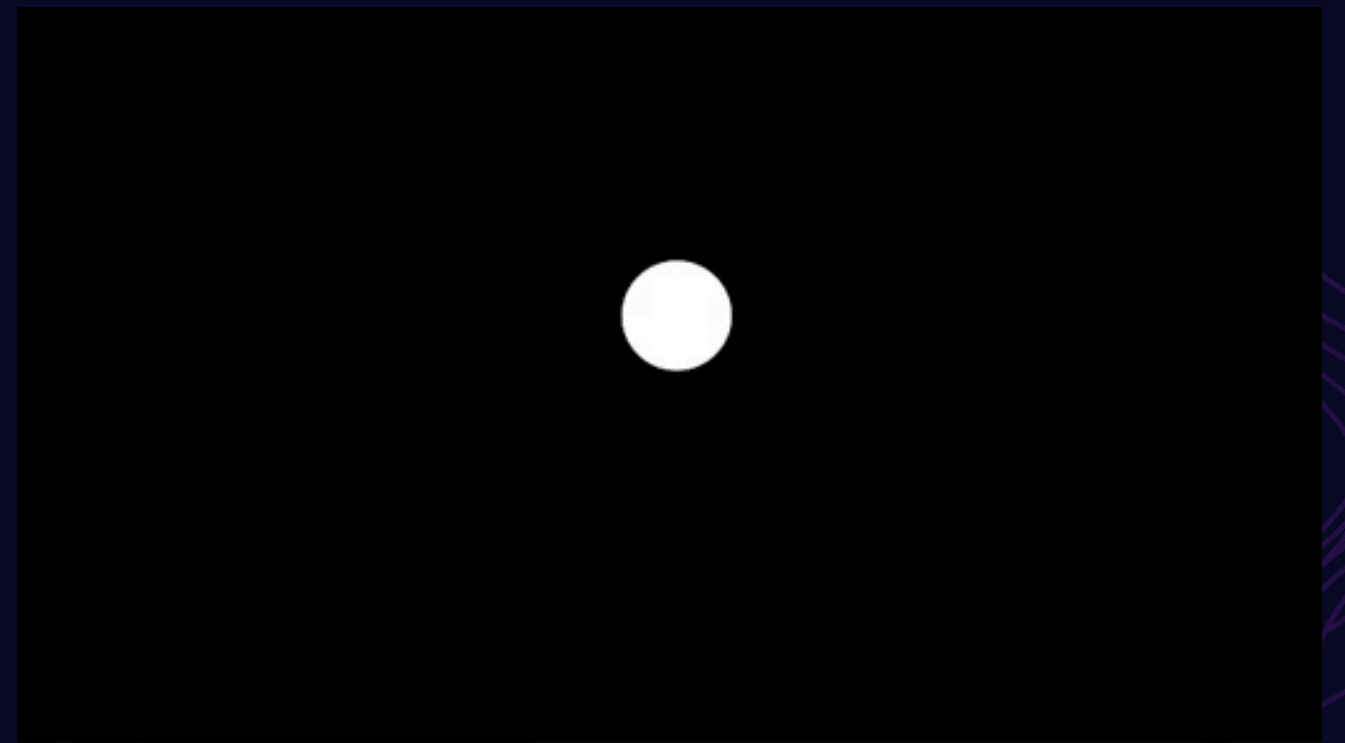
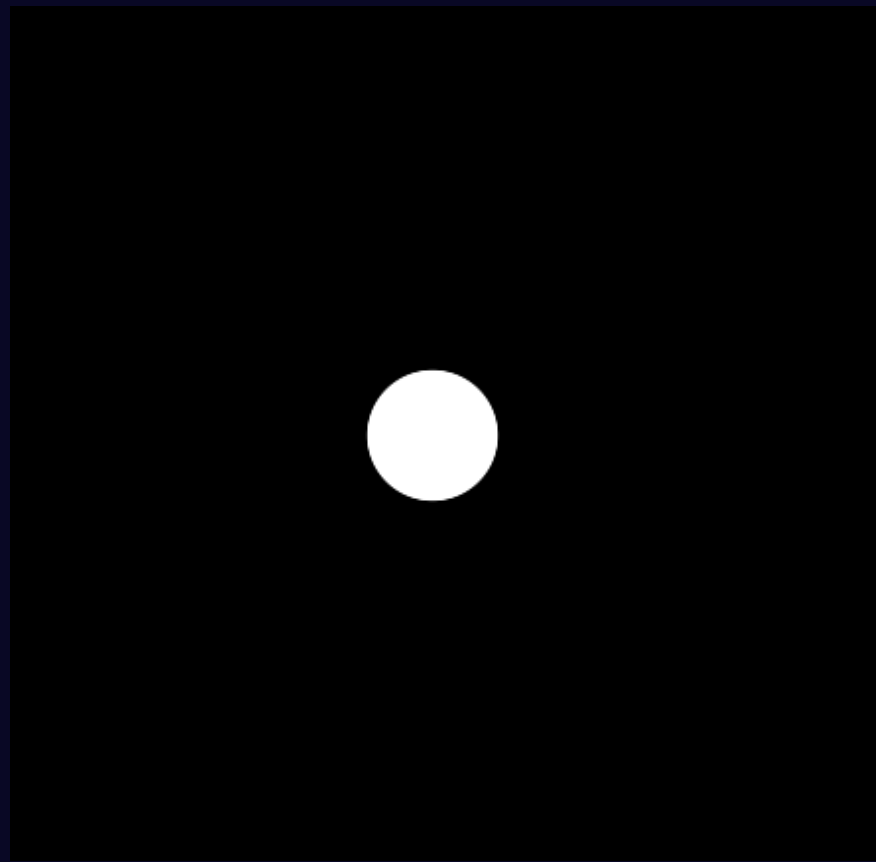
BABY, BABY ROTATION

BABY ROTATION

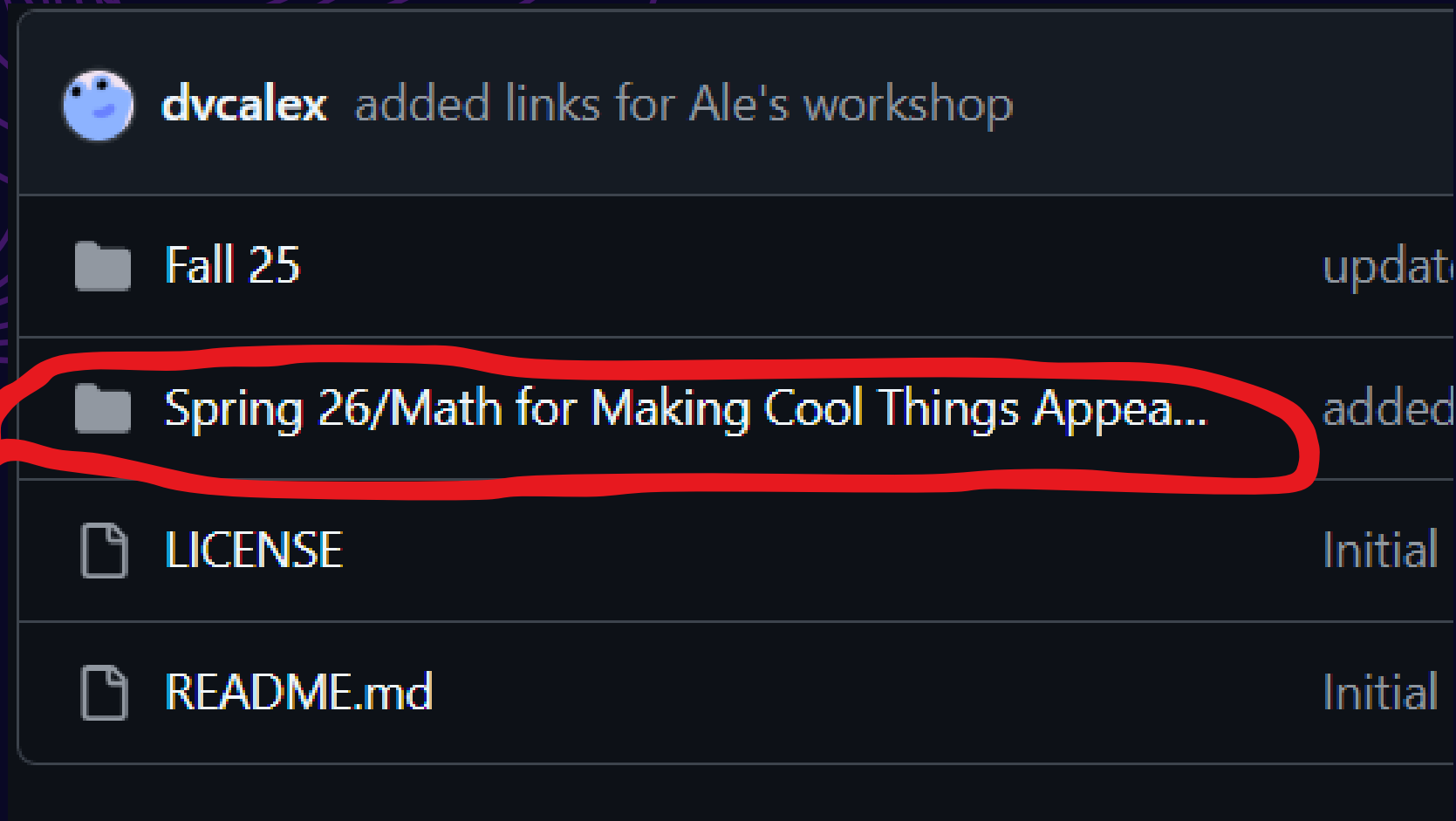
What I say?.... TRANSFORMATION STYLE!!!!



**LET'S CODE
SOMETHING COOL**



<https://github.com/GraphicsProgrammingKnights/Workshops/tree/main>
also putting this on math channel



 dvcalex added links for Ale's workshop		
	Fall 25	update
	Spring 26/Math for Making Cool Things Appea...	added
	LICENSE	Initial
	README.md	Initial

select this one

copy the link for the one we
are doing

shadertoy.com

HERE IS THE FINAL CODE

Image

Shader Inputs

```
1 float circle(vec2 uv, float r, float blur)
2 {
3     float d = length(uv);
4     return smoothstep(r, r - blur, d);
5 }
6
7 // physics
8 float gravity = 9.8;      // downward acceleration
9 float bounceLoss = 0.85;  // energy kept after bounce
10 float startHeight = 1.0; // initial drop height
11 void mainImage(out vec4 fragColor, in vec2 fragCoord)
12 {
13
14     vec2 uv = fragCoord / iResolution.xy;
15     uv = uv * 2.0 - 1.0;
16     float aspect = iResolution.x / iResolution.y;
17     uv.x *= aspect;
18
19     float radius = 0.15;
20     float blur = 2.0 / iResolution.y;
21
22     float time = iTime;
23     float height = startHeight;
24     float velocity = 0.0;
```

<https://www.shadertoy.com/view/3XyBzR>

2/15/26, 10:01 PM

basketball|||||||

```
25     float y = startHeight;
26
27     for(int i = 0; i < 1000; i++)
28     {
29         float fallTime = sqrt(2.0 * height / gravity);
30
31         if(time < fallTime)
32         {
33             y = height - 0.5 * gravity * time * time;
34             break;
35         }
```

```
36
37         time -= fallTime;
38
39         velocity = gravity * fallTime;
40         velocity *= bounceLoss;
41         height = (velocity * velocity) / (2.0 * gravity);
42         float riseTime = velocity / gravity;
43         if(time < riseTime)
44         {
45             y = velocity * time - 0.5 * gravity * time * time;
46             break;
47         }
48
49         time -= riseTime;
50     }
51     // place ball (centered horizontally)
52     vec2 center = vec2(0.0, -1.0 + radius + y);
53     // object space
54     vec2 local = uv - center;
55     float ball = circle(local, radius, blur);
56     vec3 color = vec3(ball); // white
57     fragColor = vec4(color, 1.0);
58 }
```


WRAP UP EXAMPLE 1 (GAMES)

KUXIR PINCH (ROCKET LEAGUE)



WHAT HAPPENS HERE?

Forces from the vectors in the car and wall collide

The angle creates a tight window in which the vector can execute

The momentum changes drastically and reflects at the speed the ball goes

The game uses every frame of this to correctly analyze the function of the ball and wall interacting

WRAP UP EXAMPLE 2 (ML)

ELEVENLABS API



ElevenLabs and most voice agents utilize matrices to determine certain characteristics about the voice

They store these recordings and with the data from the matrix, the AI can determine how to manipulate the voice to sound according to what you want

These manipulations can also be seen as vectors even, as how a certain direction goes can determine age or gender of the voice

WRAP UP EXAMPLE 3 (GRAPHICS)

CARS (MOVIE)

The Cars movie completely revolutioned my whole life and math was a key part as to why the movie was amazing

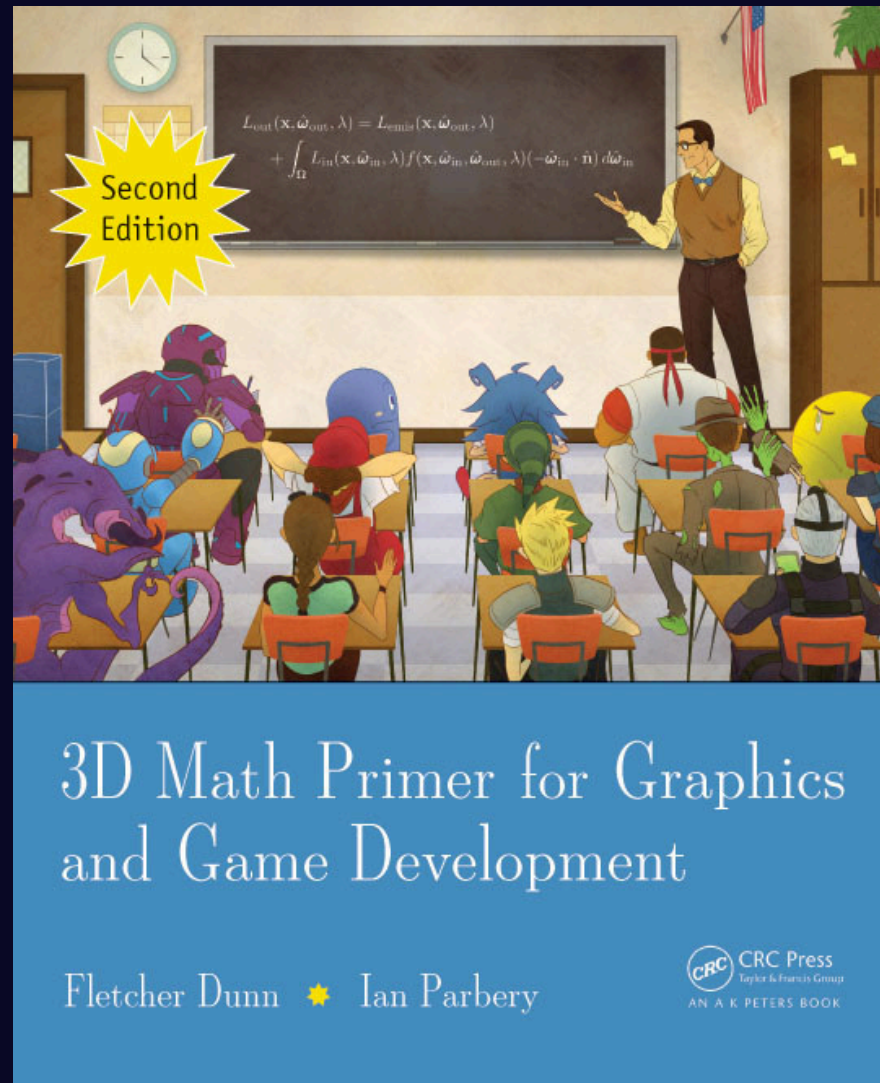
They use transformations that were advancing with Pixar's animation and there is an article online talking about how Ray Tracing was utilized

In terms of matrices, it was also a great hit apart from the Ray Tracing utilized as they were key to determine positioning and scale



Great resources (and why)

Beginner



Learn computer graphics from scratch and for free.

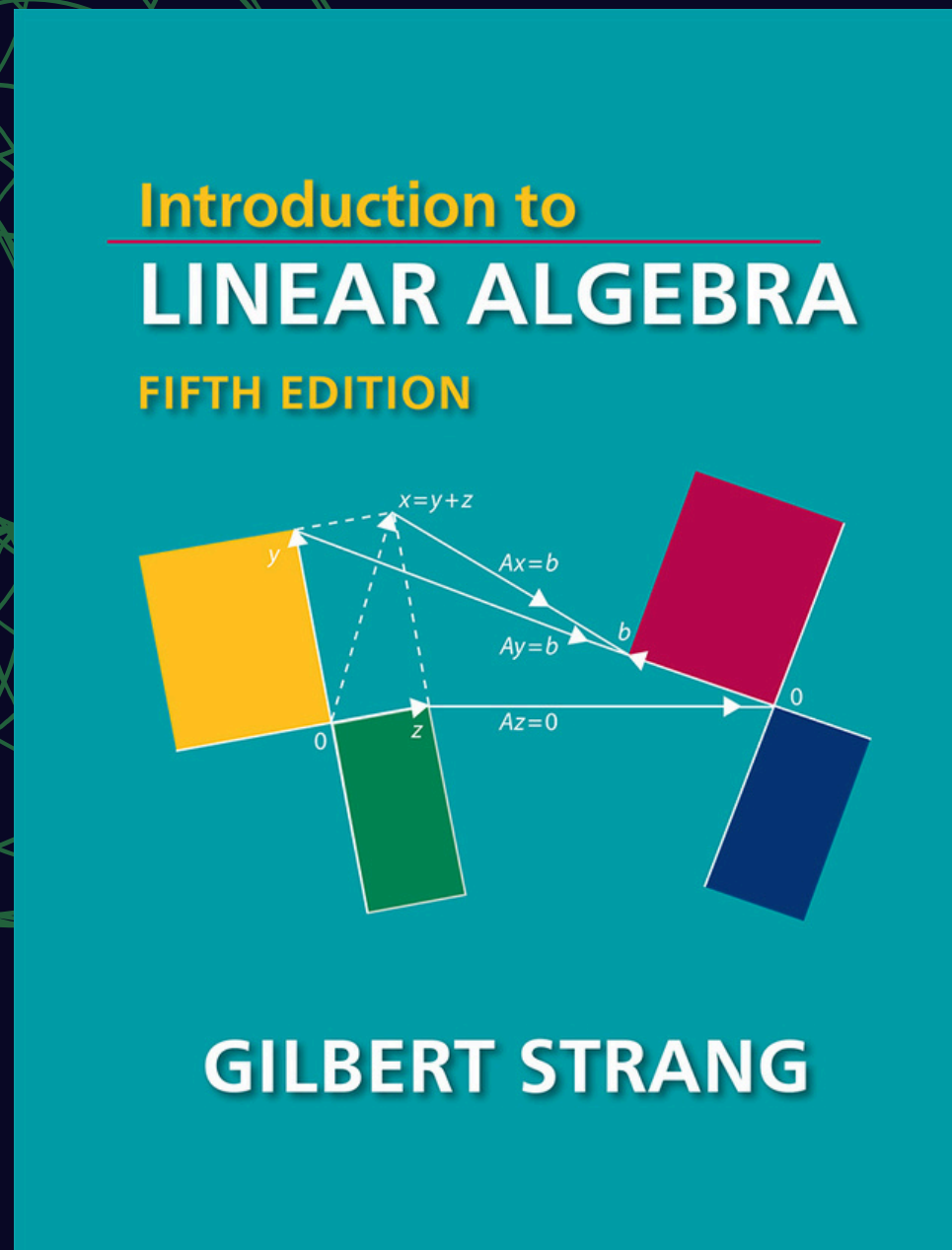
**The usual beloved lessons.
A blog, some private courses,
and an upcoming book project!**

[HTTPS://GAMEMATH.COM/BOOK/INTRO.HTML](https://gamemath.com/book/intro.html)

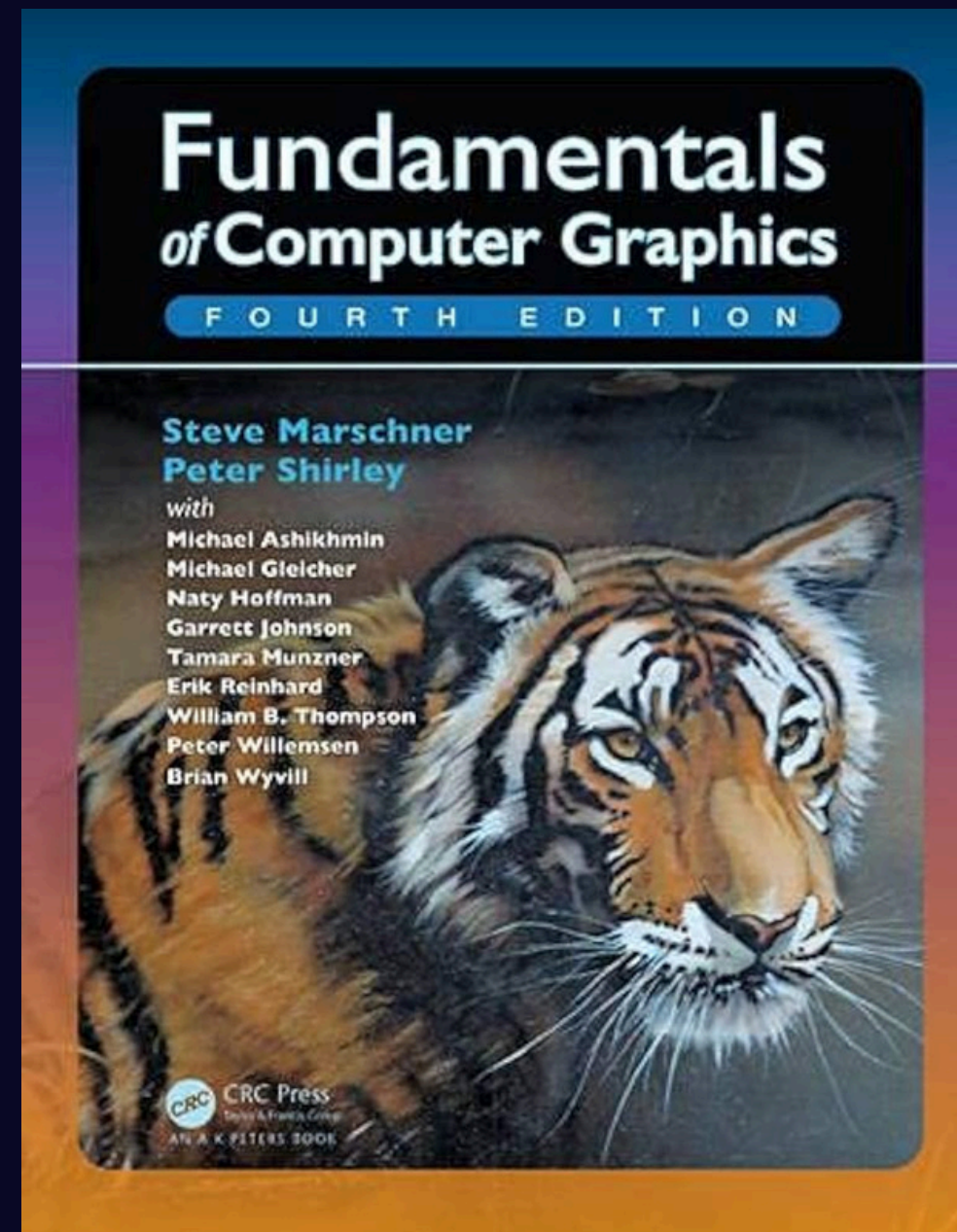
[HTTPS://WWW.SCRATCHAPIXEL.COM/](https://www.scratchapixel.com/)

Great resources (and why)

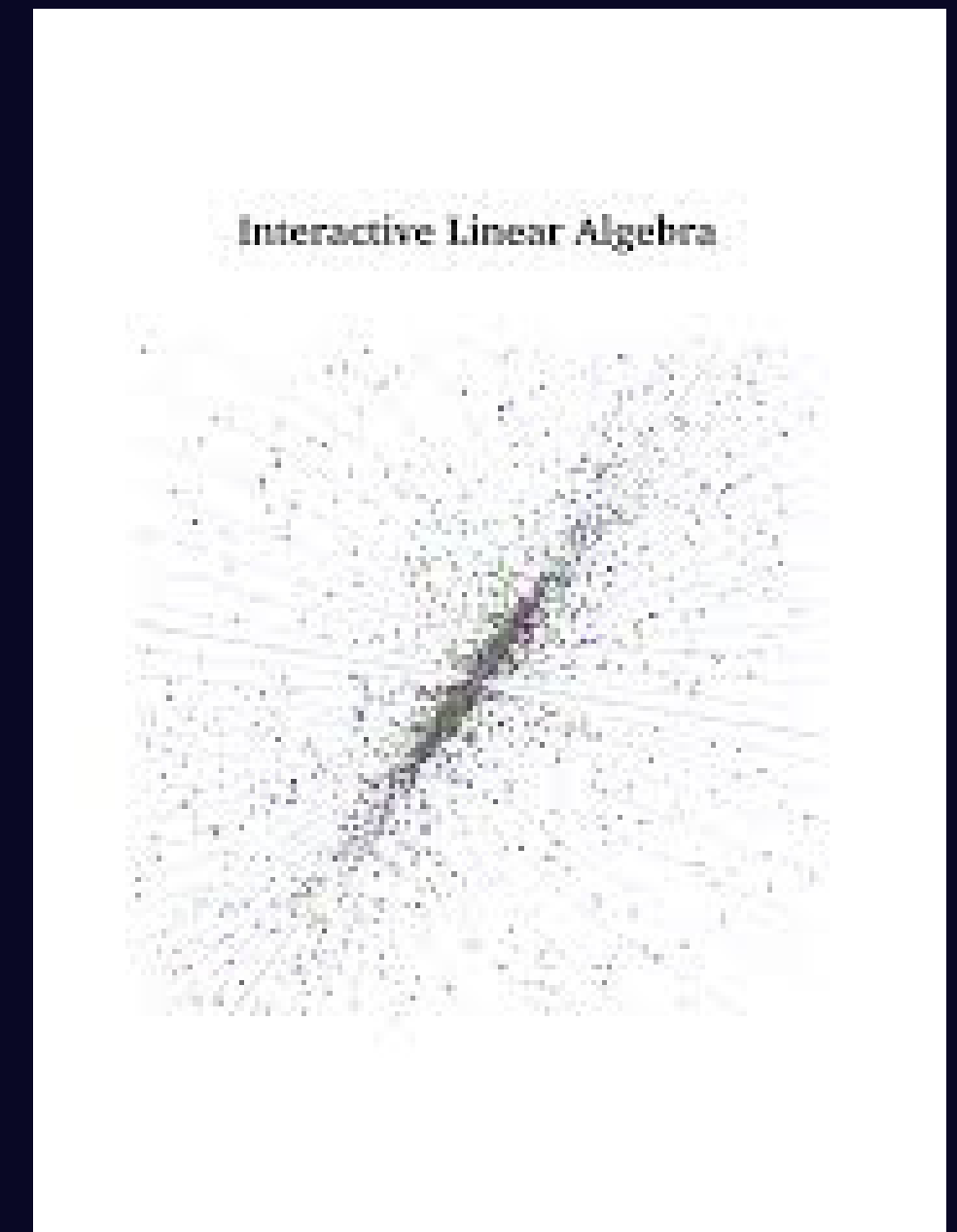
Intermediate



MIT OCW LIN ALGEBRA



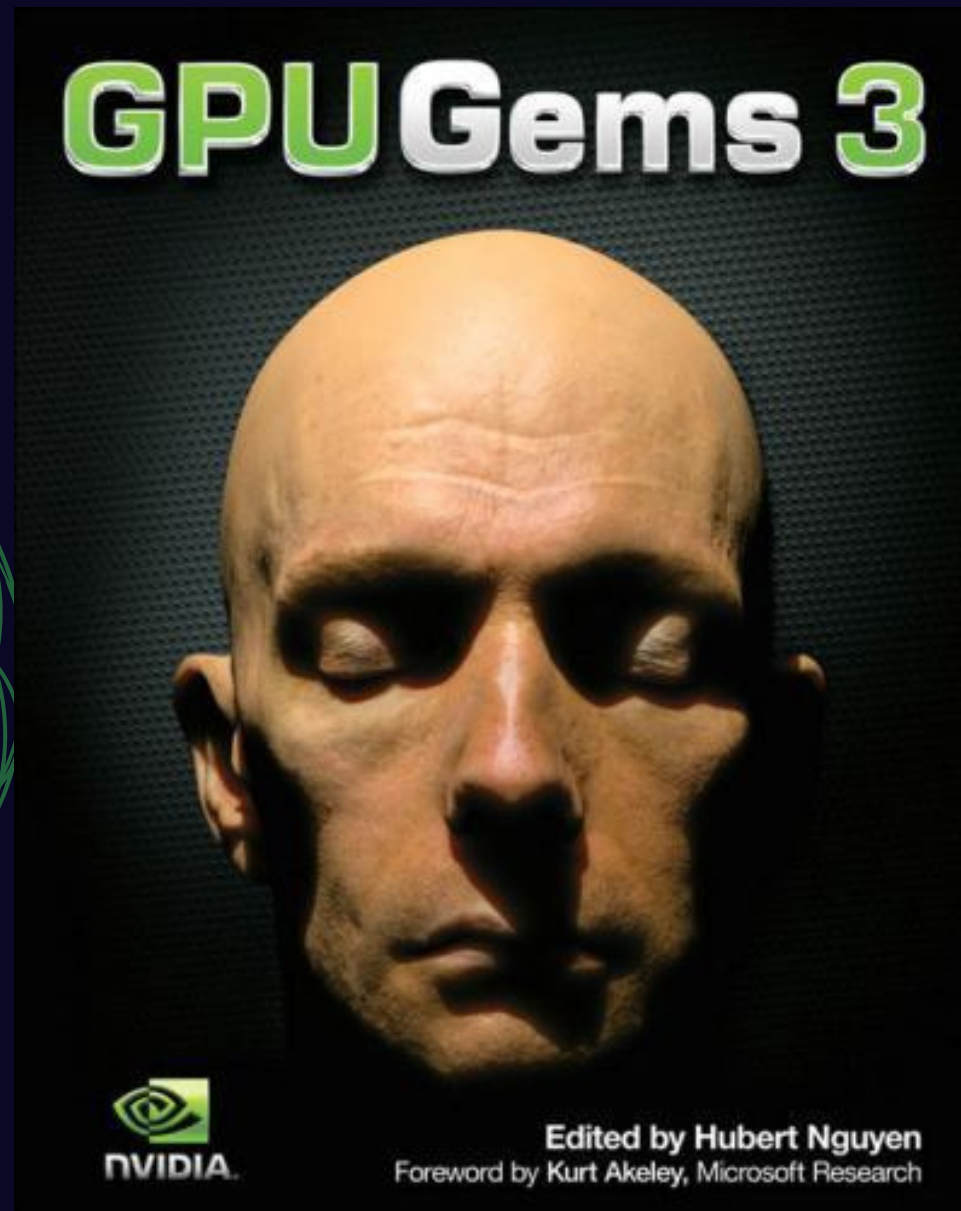
FUNDAMENTALS OF CG BOOK



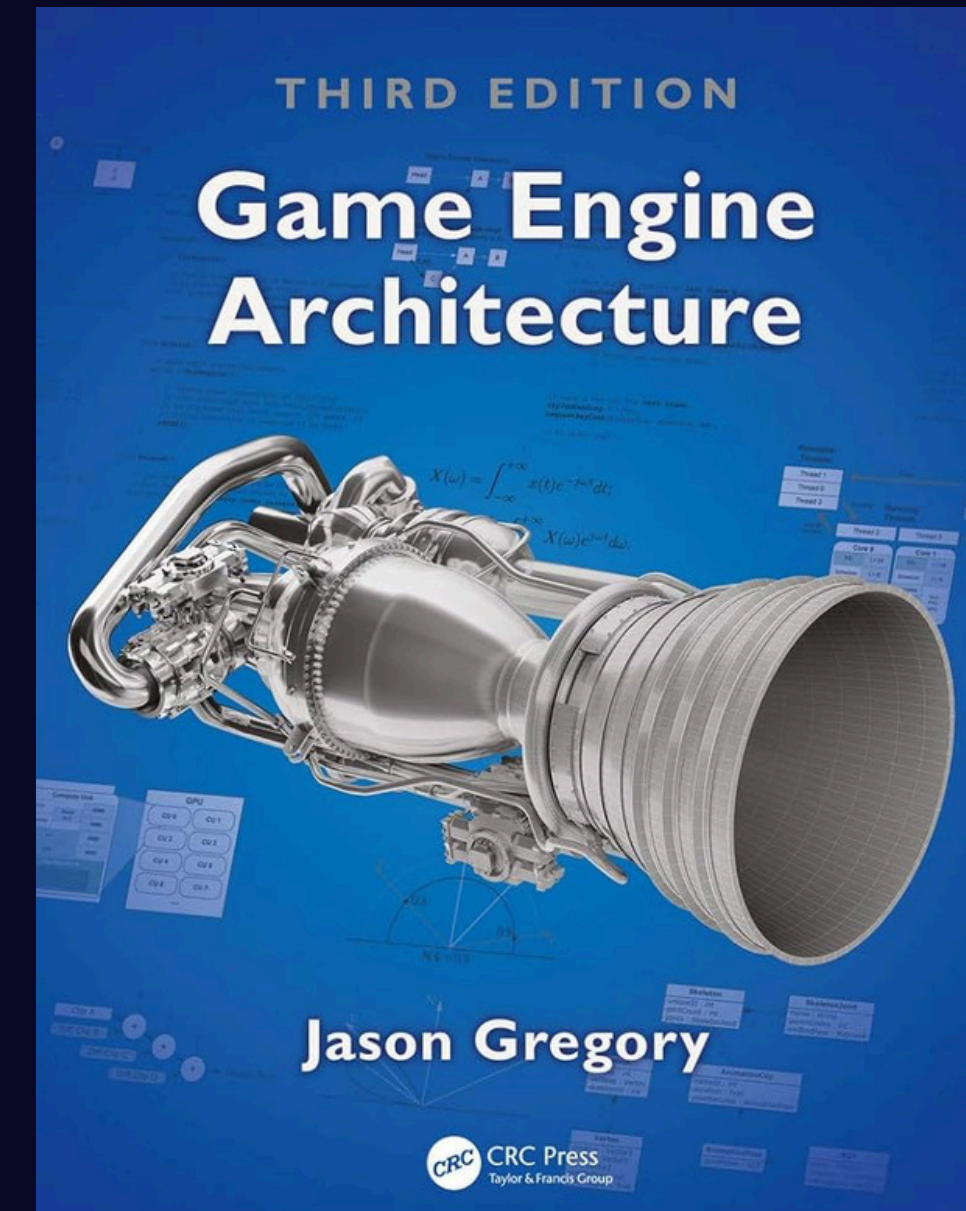
INTERACTIVE LINEAR ALGEBRA

Great resources (and why)

Hard



GPU GEMS (ANY EDITION)



GAME ENGINE ARCHITECTURE

QUESTIONS?



Feedback
Fill this form out!



Follow our Socials!

- Stay up to date on our events by following our Instagram!
- Rewatch GPK meetings at your own pace when published on YouTube!



Thank you!

